Chatbot Trabalho Garantido

ESTRUTURA DO PROJETO

Apresentação do projeto

Desenvolveremos um chatbot capaz de responder perguntas sobre o edital do programa Trabalho Garantido da FMU. Utilizaremos técnicas de Processamento de Linguagem Natural, embeddings, busca semântica e um modelo de linguagem para gerar respostas baseadas no documento.


PDF
Edital em PDF
Fonte de conhecimento
Preparação dos textos
Chunking e embeddings
Banco vetorial Chroma
Armazenamento e busca semântica
LLM via Groq
Geração de respostas com contexto
Streamlit
Interface web do chatbot

Criar os arquivos do projeto

- Criar a pasta do projeto
- Entre no vscode e abra a pasta criada
- No Explorer do VS Code, crie os seguintes arquivos:
  - app.py (código principal do projeto)
  - requirements.txt (lista das bibliotecas necessárias)
  - .env (armazenamento local da chave API)
  - EDITAL-Trabalho-Garantido.pdf (base de conhecimento utilizada pelo chatbot)

Preparando o ambiente

- Criar um ambiente virtual:
  - Abra o terminal: ctrl ‘
  - python -m venv .venv
- Ativar o ambiente virtual
  - .\.venv\Scripts\Activate.ps1
- Após a ativação deverá aparecer:
  - (.venv) PS C:\...\chatbot_TG>

Preparar o requirements.txt

- Abra o arquivo requirements e adicione:
    - streamlit
    - python-dotenv
    - langchain-community
    - langchain-text-splitters
    - langchain-chroma
    - langchain-groq
    - langchain-huggingface
    - pypdf
    - sentence-transformers

Preparando o ambiente

- Se o PowerShell bloquear a execução do script, podemos usar o terminal Command Prompt do VS Code e executar:
  - .venv\Scripts\activate.bat
- Selecionar o interpretador no VS Code:
  - Pressione Ctrl + Shift + P e procure: Python: Select Interpreter

Instalar as bibliotecas

- Com o ambiente virtual ativado, execute:
  - python -m pip install -r requirements.txt

Configurar a chave da API Groq

- O chatbot irá acessar um modelo de linguagem pelo groq
  - https://console.groq.com/keys
- Conteúdo do arquivo .env:
  - GROQ_API_KEY=gsk_sua_chave_aqui

Testar o ambiente com Streamlit

- Antes de começar o chatbot, vamos verificar se o Streamlit está funcionando.
- No arquivo app.py escreva:

```
import streamlit as st

st.title("Chatbot Trabalho Garantido")
st.write("Projeto de Processamento de Linguagem Natural")
st.success("Ambiente configurado com sucesso!")
```

- Salve o arquivo e execute no terminal:

```
python -m streamlit run app.py
```

Tokens

Os modelos de linguagem, como o GPT-OSS-120B que estamos utilizando pela Groq, não processam diretamente as palavras como nós as enxergamos.
Eles trabalham com tokens, que são unidades menores de texto. Um token pode representar uma palavra inteira, parte de uma palavra ou até um sinal de pontuação.

Por exemplo, a frase:

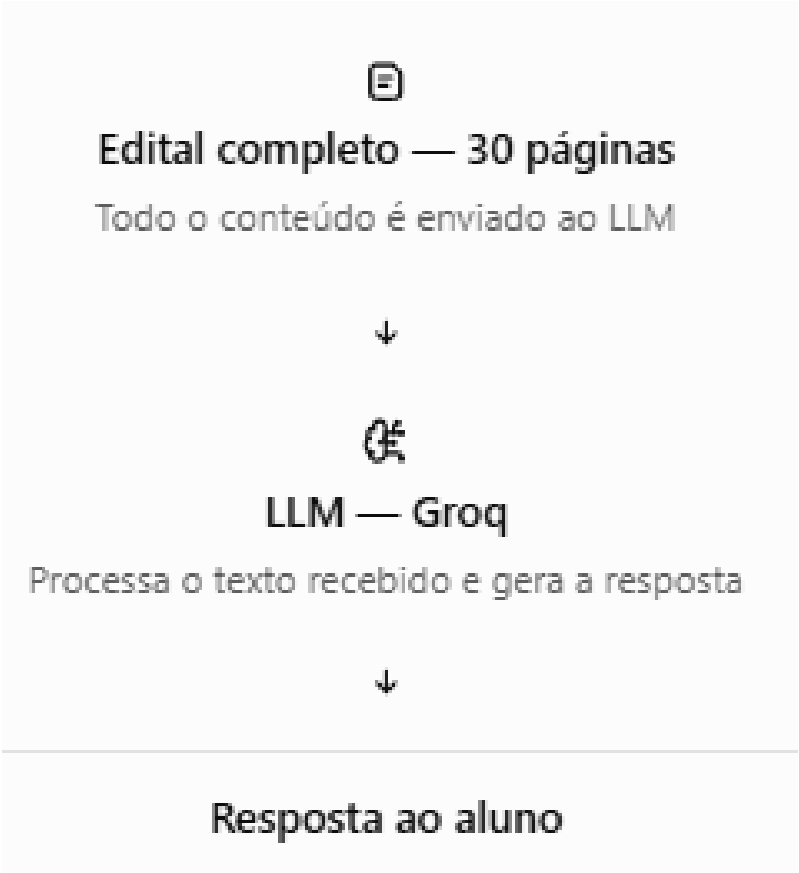
O aluno poderá participar do programa Trabalho Garantido

Será dividida em tokens pelo tokenizador do modelo. Essa divisão não corresponde necessariamente a uma palavra por token. Quanto maior o texto enviado ao modelo, maior será a quantidade de tokens utilizados.

Tokens

O que acontece se enviarmos o edital inteiro?
Imagine que nosso edital tenha 30 páginas. Quando um aluno perguntar:

Qual é o prazo para participar do Trabalho Garantido? Poderíamos enviar ao modelo:



Tokens

O problema é que o modelo receberia todas as páginas, mesmo que a resposta estivesse em apenas um parágrafo.

Isso pode causar três problemas principais:

- Maior consumo de tokens: o texto completo precisa ser processado a cada pergunta.
- Maior custo ou consumo de limites da API: dependendo do serviço e do plano, tokens podem ser cobrados ou sujeitos a limites de utilização.
- Menor eficiência: o modelo recebe muitas informações desnecessárias, o que pode aumentar a latência e dificultar a localização da informação relevante.

Além disso, cada modelo possui uma janela de contexto, isto é, um limite para a quantidade de tokens que consegue processar em uma solicitação.

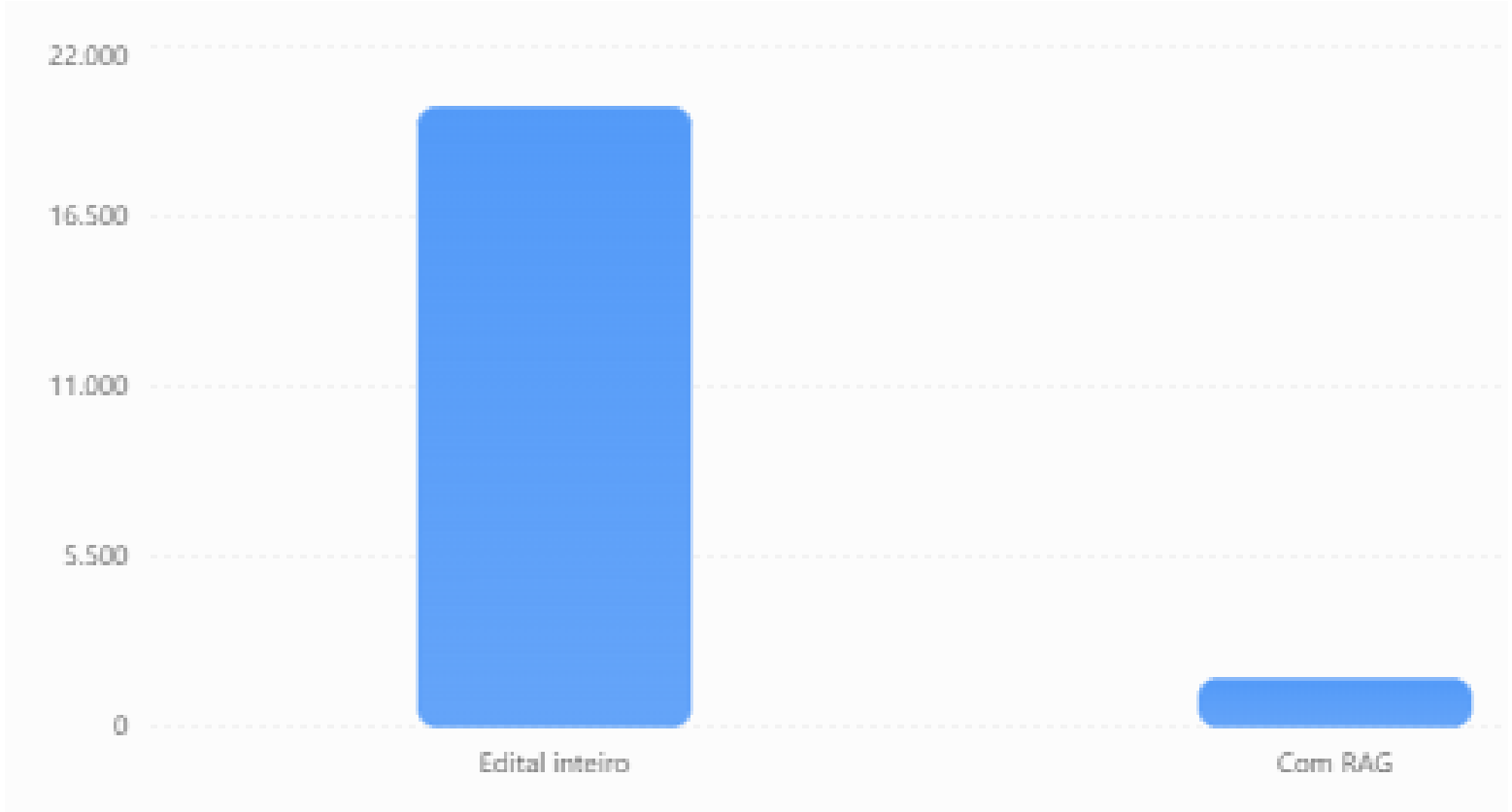
Como o RAG resolve esse problema?

- Em vez de enviar o edital inteiro, nosso sistema primeiro divide o documento em pequenos trechos, chamados **chunks**.
- Quando o aluno faz uma pergunta, o sistema procura os trechos mais relacionados ao assunto e envia somente esses trechos ao LLM.



Comparação de consumo

Vamos imaginar que o edital completo tenha 20.000 tokens e que o código esteja configurado para recuperar 5 trechos e que esses trechos recuperados tenham, juntos, aproximadamente 1.500 tokens.



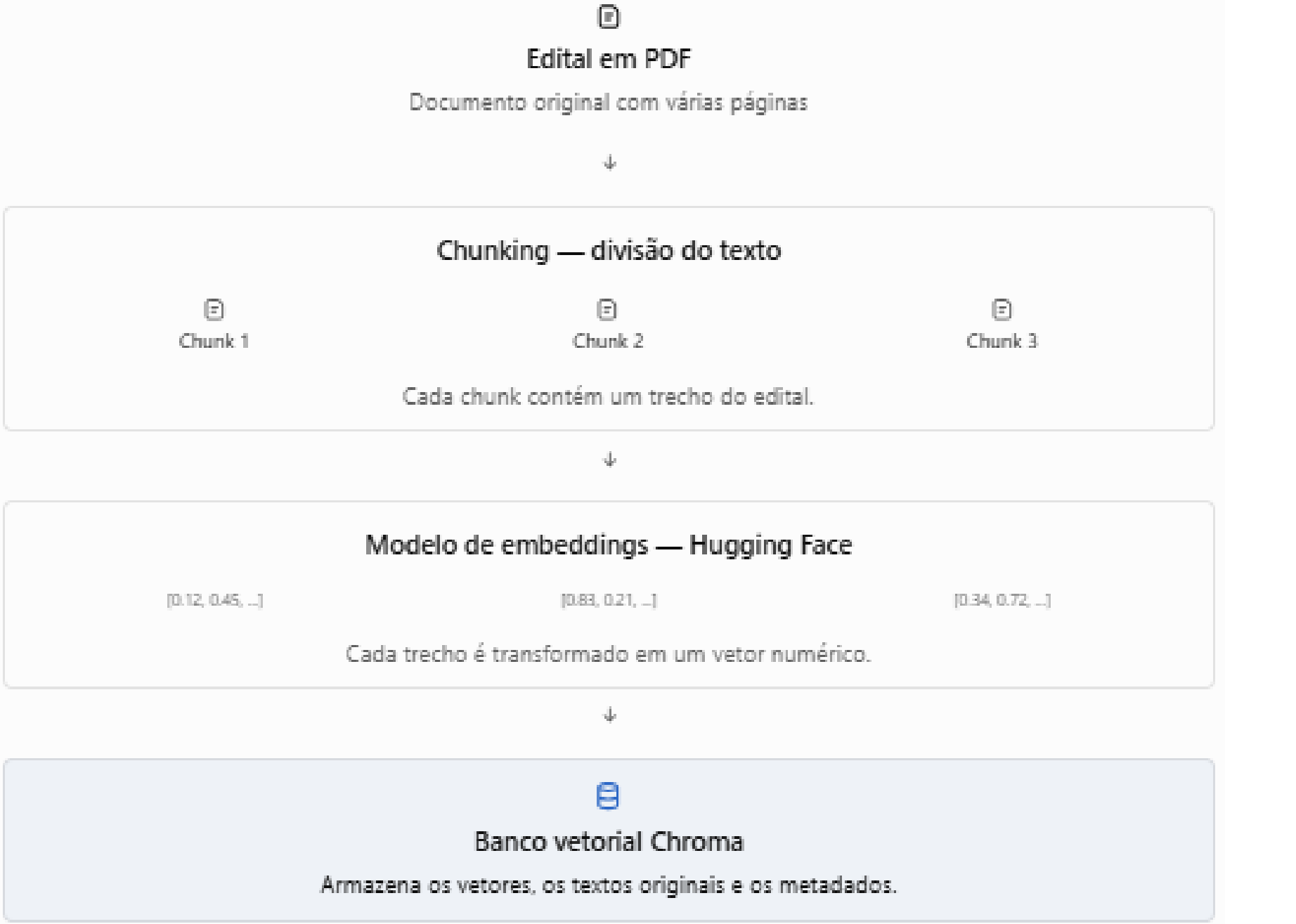
Nesse exemplo, o RAG reduziria em 92,5% a quantidade de tokens do documento enviados ao LLM.

Embeddings

- E os embeddings também consomem tokens?
- O texto também passa por um processo de tokenização, mas não estamos enviando esses textos para a API da Groq.
- Nesse caso, utilizamos um modelo do **Hugging Face**, executado localmente, para transformar os chunks em vetores numéricos, portanto, sem consumo de tokens.

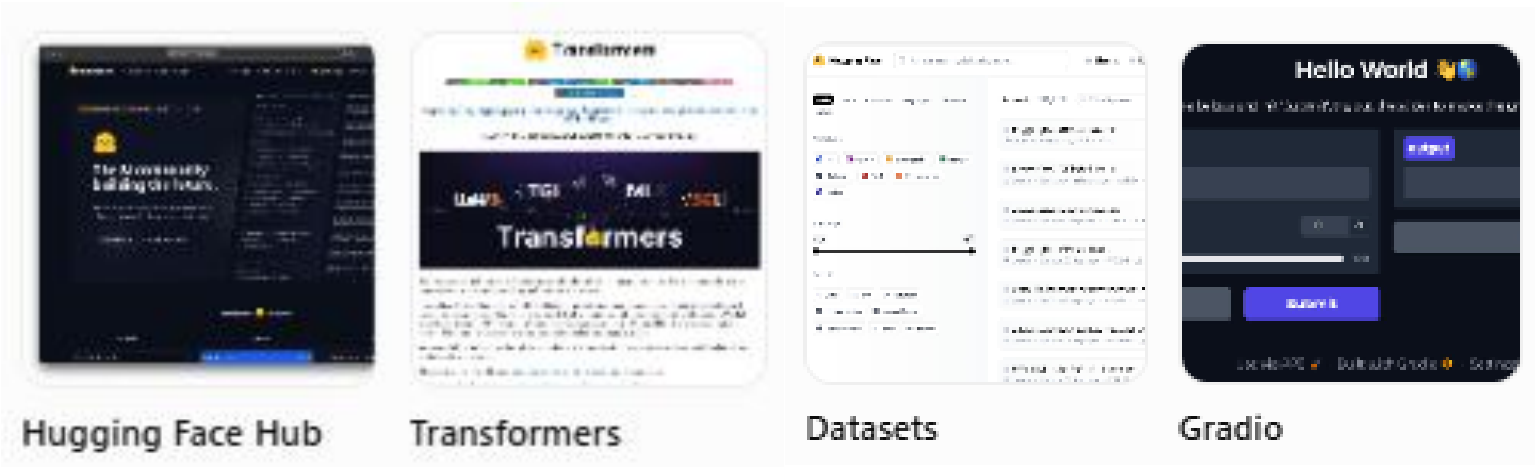
Processo	Tecnologia	Finalidade
Gerar embeddings	Hugging Face	Representar os textos numericamente para a busca semântica
Gerar respostas	GPT-OSS-120B via Groq	Interpretar o contexto recuperado e responder à pergunta

O primeiro exige processamento computacional local. O segundo consome tokens da API da Groq.


Edital em PDF
Documento original com várias páginas
Chunking — divisão do texto
Chunk 1
Chunk 2
Chunk 3
Cada chunk contém um trecho do edital.
Modelo de embeddings — Hugging Face
[0.12, 0.45, ...]
[0.83, 0.21, ...]
[0.34, 0.72, ...]
Cada trecho é transformado em um vetor numérico.
Banco vetorial Chroma
Armazena os vetores, os textos originais e os metadados.

Hugging Face (https://huggingface.co/)

- A **Hugging Face** é uma organização que desenvolve uma plataforma para compartilhar, treinar e implantar modelos de inteligência artificial, além de manter ferramentas amplamente usadas pela comunidade de IA. Tornou-se uma das principais referências do ecossistema de **IA de código aberto**, reunindo milhares de modelos, conjuntos de dados e aplicações colaborativas. Podemos dizer que é o GitHub da IA. Principais componentes:
- A plataforma reúne diversos serviços que formam um ecossistema integrado para pesquisa e desenvolvimento em IA.



Modelos **bibliotecas Python** **bases** **web**

Modelo do Hugging Face

Escolhemos o **sentence-transformers/all-MiniLM-L6-v2**, que é um modelo relativamente pequeno e adequado para demonstrar o funcionamento de embeddings sem exigir uma GPU dedicada.

Ele transforma cada texto em um vetor de 384 dimensões.

Perguntas de exemplo

- Qual é o objetivo do Programa Trabalho Garantido?
- Quem pode participar do programa?
- Quais são as condições para participar?
- O que acontece se o aluno não conseguir emprego?

Arquitetura de Software no projeto Trabalho Garantido

Etapa do projeto		Relação com a arquitetura
1	**Configuração da API Groq**	Integração com o serviço externo de IA e gerenciamento das credenciais de acesso.
2	**Interface Streamlit**	Camada de apresentação, responsável pela interação com o usuário.
3	**Carregamento do PDF**	Definição da entrada e leitura dos documentos.
4	**Divisão em chunks**	Organização e preparação dos textos para recuperação posterior.
5	**Modelo de embeddings**	Representação numérica e semântica dos textos.
6	**Banco vetorial Chroma**	Armazenamento e pesquisa vetorial dos documentos processados.
7	**Retriever**	Mecanismo de recuperação dos trechos mais relevantes.
8	**LLM via Groq**	Componente responsável pela geração das respostas com base no contexto recuperado.
9	**Preparação do RAG**	Integração dos componentes e organização do processamento do sistema.
10	**Inicialização do chatbot**	Preparação e reutilização dos recursos da aplicação.
11	**Histórico de mensagens**	Gerenciamento do estado da conversa durante a sessão.
12	**Pergunta do usuário**	Início do fluxo de comunicação entre usuário, recuperação de informações e LLM.
13	**Resposta do chatbot**	Exibição da resposta gerada, concluindo o fluxo de comunicação.